



Documentation Complète JobbingTrack v4.1



Vue d'ensemble

Documentation exhaustive du système JobbingTrack incluant architecture, APIs, déploiement, développement et guides pratiques.



Table des matières

-  [Documentation JobbingTrack](#)
 -  [Navigation Documentation - JobbingTrack](#)
 -  [Architecture Microservices - JobbingTrack](#)
 -  [Services Microservices - JobbingTrack](#)
 -  [Base de Données - JobbingTrack](#)
 -  [API Reference - JobbingTrack](#)
 -  [Endpoints API - JobbingTrack](#)
 -  [Démarrage Rapide - JobbingTrack](#)
 -  [Déploiement Production - JobbingTrack](#)
 -  [Sécurité Déploiement - JobbingTrack](#)
 -  [Configuration Développement - JobbingTrack](#)
 -  [Workflow Développement - JobbingTrack](#)
 -  [Tests et Qualité - JobbingTrack](#)
 -  [Guide Frontend - JobbingTrack](#)
 -  [Guide Mobile - JobbingTrack](#)
 -  [Guide Administration - JobbingTrack](#)
 -  [Guide Dépannage - JobbingTrack](#)
 -  [Guide Performance - JobbingTrack](#)
 -  [Guide Sécurité - JobbingTrack](#)
-

Documentation JobbingTrack

Source: README.md

Vue d'ensemble

Documentation complète et organisée du projet **JobbingTrack v4.1** - Système de suivi de candidatures avec architecture microservices, dashboard administrateur et applications mobiles.

Navigation complète - Guide de navigation dans toute la documentation

Structure de la documentation

docs/	
└─  README.md	# Ce fichier - index principal
└─  core/	# Documentation technique de base
└─ architecture.md	# Architecture microservices
└─ services.md	# Détail des microservices
└─ database.md	# Structure base de données
└─  api/	# Documentation API
└─ api-reference.md	# Guide API complet
└─ endpoints.md	# Liste des endpoints
└─  deployment/	# Guides de déploiement
└─ getting-started.md	# Démarrage rapide
└─ production.md	# Déploiement production
└─ security.md	# Sécurité déploiement
└─  development/	# Guides développement
└─ setup.md	# Configuration environnement
└─ workflow.md	# Workflow développement
└─ testing.md	# Stratégies de tests
└─  frontend/	# Guide frontend
└─ guide.md	# Développement frontend
└─  mobile/	# Guide mobile
└─ guide.md	# Développement mobile
└─  administration/	# Guide administration
└─ guide.md	# Dashboard administrateur
└─  troubleshooting/	# Dépannage
└─ guide.md	# Guide de résolution
└─  performance/	# Optimisation
└─ guide.md	# Guide performance
└─  security/	# Sécurité
└─ guide.md	# Guide sécurité

Documentation principale

Architecture et Infrastructure

- [Architecture Microservices](#) - Vue complète de l'architecture
- [Services Backend](#) - Détail des 18+ microservices
- [Base de Données](#) - Schema PostgreSQL et relations

API et Intégration

- [API Reference](#) - Documentation complète des APIs
- [Endpoints](#) - Liste exhaustive des endpoints

Déploiement

- [Démarrage Rapide](#) - Installation et configuration
- [Production](#) - Déploiement en production
- [Sécurité](#) - Configuration sécurité

Développement

- [Configuration](#) - Environnement de développement
- [Workflow](#) - Processus de développement
- [Tests](#) - Stratégies et outils de test

Nouveautés v4.1

Base de données étendue

- **Historique des statuts** : Suivi complet des changements de statut des candidatures
- **Système de notifications** : Multi-canaux avec métadonnées et liens vers entités
- **Calendrier intégré** : Événements polymorphes liés à tous les modules
- **Synchronisation mobile** : Queue pour la fonctionnalité offline
- **Relations many-to-many** : Contacts multi-entreprises et multi-candidatures

Nouveaux modèles

- `ApplicationStatusHistory` - Historique des changements de statut
- `Notification` - Système de notifications
- `Event` - Calendrier avec relations polymorphes
- `SyncQueue` - Synchronisation mobile/offline

Nouvelles relations

- Contact ↔ Entreprise (many-to-many)
- Contact ↔ Candidature (many-to-many)
- Contact ↔ Relance (many-to-many)
- Contact ↔ Entretien (many-to-many)
- Contact ↔ Événement (many-to-many)

Démarrage rapide

1. Consulter l'architecture : [core/architecture.md](#)
2. Comprendre la base de données : [core/database.md](#)

3. Explorer les APIs : [api/api-reference.md](#)

4. Configurer le développement : [development/setup.md](#)

Services disponibles

Services Backend (18+)

- **API Gateway** - Point d'entrée unique (Port: 3000)
- **Auth Service** - Authentification et autorisation (Port: 3001)
- **Application Service** - Gestion des candidatures (Port: 3002)
- **Company Service** - Gestion des entreprises (Port: 3003)
- **Contact Service** - Gestion des contacts (Port: 3004)
- **Interview Service** - Gestion des entretiens (Port: 3005)
- **Call Service** - Gestion des appels (Port: 3006)
- **Event Service** - Gestion des événements (Port: 3007)
- **Followup Service** - Gestion du suivi (Port: 3008)
- **Profile Service** - Gestion des profils (Port: 3009)
- **Notification Service** - Système de notifications (Port: 3010)
- **Workflow Service** - Gestion des workflows (Port: 3011)
- **Dashboard Service** - Dashboard administrateur (Port: 3012)
- **Security Service** - Service de sécurité (Port: 3013)
- **Metrics Service** - Métriques système (Port: 3014)
- **Deployment Service** - Service de déploiement (Port: 3015)
- **Docker Stats Service** - Statistiques Docker (Port: 3016)
- **Scheduler Service** - Planification (Port: 3017)

Services Infrastructure

- **PostgreSQL** - Base de données principale (Port: 5432)
- **Redis** - Cache et sessions (Port: 6379)
- **Prometheus** - Monitoring et métriques
- **Grafana** - Visualisation des métriques
- **cAdvisor** - Monitoring containers (Port: 8081)

Applications

Frontend

- **Next.js** - Interface web moderne (Port: 8080)
- **Admin Dashboard** - Interface d'administration complète
- **Responsive Design** - Compatible mobile et desktop

Mobile

- **Flutter** - Application mobile cross-platform
- **Offline Support** - Fonctionnement hors ligne
- **Synchronisation** - Sync bidirectionnelle avec le backend

Migration depuis v4.0

Les utilisateurs de la version 4.0 doivent :

1. **Sauvegarder** leur base de données actuelle
2. **Appliquer la migration** : `cd backend && npx prisma migrate dev`
3. **Mettre à jour** les services backend pour utiliser les nouvelles relations
4. **Tester** les nouvelles fonctionnalités

Support

- **Issues** : Créer une issue sur GitHub
- **Documentation** : Toutes les mises à jour sont synchronisées avec les PDFs
- **Migration** : Guide de migration disponible dans chaque document
- **PDF Complet** : [documentation-complete.pdf](#)

Version : 4.1 - Base de données étendue **Dernière mise à jour** : \$(date +%Y-%m-%d) **Équipe** :
JobbingTrack Development Team

Navigation Documentation - JobbingTrack

Source: *navigation.md*

Navigation principale

Documentation du Projet

-  [Accueil](#) |  [Documentation Centralisée](#)

Démarrage Rapide

-  [Installation](#) |  [Configuration Développement](#)

Architecture et Infrastructure

-  [Architecture Microservices](#) |  [Services](#) |  [Base de Données](#)

API et Intégration

-  [API Reference](#) |  [Endpoints](#)

Déploiement

-  [Démarrage](#) |  [Production](#) |  [Sécurité](#)

Développement

-  [Configuration](#) |  [Workflow](#) |  [Tests](#)

Applications

-  [Frontend](#) |  [Mobile](#)

Administration

-  [Guide Administration](#) |  [Dépannage](#)

Performance et Sécurité

-  [Performance](#) |  [Sécurité](#)

Structure des dossiers

docs/	
└─  README.md	# Index principal
└─  navigation.md	# Ce fichier de navigation
└─  core/	# Documentation technique de base
└─ architecture.md	# Architecture microservices
└─ services.md	# Détail des microservices
└─ database.md	# Structure base de données
└─  api/	# Documentation API
└─ api-reference.md	# Guide API complet
└─ endpoints.md	# Liste des endpoints
└─  deployment/	# Guides de déploiement
└─ getting-started.md	# Démarrage rapide
└─ production.md	# Déploiement production
└─ security.md	# Sécurité déploiement
└─  development/	# Guides développement
└─ setup.md	# Configuration environnement
└─ workflow.md	# Workflow développement
└─ testing.md	# Stratégies de tests
└─  frontend/	# Guide frontend
└─ guide.md	# Développement frontend
└─  mobile/	# Guide mobile
└─ guide.md	# Développement mobile
└─  administration/	# Guide administration
└─ guide.md	# Dashboard administrateur
└─  troubleshooting/	# Dépannage
└─ guide.md	# Guide de résolution
└─  performance/	# Optimisation
└─ guide.md	# Guide performance
└─  security/	# Sécurité
└─ guide.md	# Guide sécurité

Liens rapides

Pour les développeurs

- [Configuration](#) - Environnement de développement
- [API](#) - Documentation des APIs
- [Architecture](#) - Vue technique
- [Tests](#) - Stratégies de tests

Pour les administrateurs

- [Administration](#) - Guide administration
- [Déploiement](#) - Production
- [Sécurité](#) - Bonnes pratiques
- [Monitoring](#) - Surveillance

Pour les utilisateurs

- [Démarrage](#) - Installation
 - [Dépannage](#) - Résolution problèmes
 - [Performance](#) - Optimisation
-



PDFs disponibles



PDF Principal

-  [Documentation Complète](#) - Tout en un (63 pages)



PDFs par catégorie

-  [Architecture](#)
-  [Services](#)
-  [Base de Données](#)



APIs

-  [API Reference](#)
-  [Endpoints](#)



Déploiement

-  [Démarrage](#)
-  [Production](#)



Développement

-  [Configuration](#)
 -  [Tests](#)
-

Recherche dans la documentation

Mots-clés principaux

- **Architecture** : microservices, scalabilité, Docker
 - **API** : REST, endpoints, authentification, JWT
 - **Déploiement** : production, sécurité, monitoring
 - **Développement** : Node.js, TypeScript, tests, CI/CD
 - **Base de données** : PostgreSQL, Prisma, schémas
 - **Mobile** : Flutter, synchronisation, offline
-

Nouveautés

Version 4.1

-  **Base de données étendue** avec relations many-to-many
 -  **Historique des statuts** des candidatures
 -  **Système de notifications** multi-canaux
 -  **Calendrier intégré** avec relations polymorphes
 -  **Synchronisation mobile** avec queue offline
 -  **Documentation restructurée** et organisée
-

Support

-  **Email** : support@jobbingtrack.com
 -  **Issues** : [GitHub Issues](#)
 -  **Documentation** : Tous les guides mis à jour
 -  **Migration** : Guide dans chaque document
-

Version : 4.1 - Navigation organisée **Dernière mise à jour** : Octobre 2025

Architecture Microservices - JobbingTrack

Source: `core/architecture.md`

Documentation complète de l'architecture technique de JobbingTrack v4.1.

[← Retour au README principal](#)

Vue d'ensemble

JobbingTrack est construit sur une **architecture microservices moderne**, conçue pour être scalable, maintenable et performante. Le système gère le suivi complet des candidatures avec un dashboard administrateur et une architecture distribuée.

Table des matières

-  [Architecture générale](#)
 -  [API Gateway](#)
 -  [Microservices](#)
 -  [Base de données](#)
 -  [Monitoring](#)
 -  [Sécurité](#)
 -  [Déploiement](#)
 -  [Applications mobiles](#)
 -  [Communication inter-services](#)
 -  [Scalabilité et performance](#)
 -  [Patterns et bonnes pratiques](#)
-

Architecture générale

Vue d'ensemble complète

```

graph TB
    subgraph "Clients"
        Web[🌐 Frontend Next.js<br/>Port: 8080]
        Mobile[📱 Flutter Mobile<br/>Port: 8090]
        API[🔌 API Gateway<br/>Port: 3000]
    end

    subgraph "Services Essentiels"
        Postgres[(🗄️ PostgreSQL<br/>Port: 5432)]
        Redis[(💾 Redis Cache<br/>Port: 6379)]
        MetricsAgg[📊 Metrics Aggregator<br/>Port: 3014]
        cAdvisor[🖥️ cAdvisor<br/>Port: 8081]
    end

    subgraph "Services Backend"
        Auth[🔑 Auth Service<br/>Port: 3001]
        Application[📄 Application Service<br/>Port: 3002]
        Company[🏢 Company Service<br/>Port: 3003]
        Contact[👥 Contact Service<br/>Port: 3004]
        Interview[🎤 Interview Service<br/>Port: 3005]
        Call[📞 Call Service<br/>Port: 3006]
        Event[📅 Event Service<br/>Port: 3007]
        Followup[🔄 Followup Service<br/>Port: 3008]
        Profile[👤 Profile Service<br/>Port: 3009]
        Notification[🔔 Notification Service<br/>Port: 3010]
        Workflow[⚙️ Workflow Service<br/>Port: 3011]
        Dashboard[📊 Dashboard Service<br/>Port: 3012]
        Security[🔒 Security Service<br/>Port: 3013]
        SystemMetrics[📈 System Metrics<br/>Port: 3018]
        Deployment[🚀 Deployment Service<br/>Port: 3016]
        DockerStats[🐳 Docker Stats<br/>Port: 3015]
    end

    subgraph "Monitoring"
        Prometheus[📊 Prometheus<br/>Port: 9090]
        Grafana[📈 Grafana<br/>Port: 3001]
    end

    %% Connexions clients
    Web --> API
    Mobile --> API
    API --> Auth
    API --> Application
    API --> Company
    API --> Contact
    API --> Interview
    API --> Call
    API --> Event
    API --> Followup
    API --> Profile

```

```
API --> Notification
API --> Workflow
API --> Dashboard
API --> Security
API --> SystemMetrics
API --> Deployment
API --> DockerStats

%% Connexions bases de données
Auth --> Postgres
Application --> Postgres
Company --> Postgres
Contact --> Postgres
Interview --> Postgres
Call --> Postgres
Event --> Postgres
Followup --> Postgres
Profile --> Postgres
Notification --> Postgres
Workflow --> Postgres
Dashboard --> Postgres
Security --> Postgres
SystemMetrics --> Postgres
Deployment --> Postgres

%% Connexions cache et monitoring
Auth --> Redis
Notification --> Redis
MetricsAgg --> cAdvisor
MetricsAgg --> Prometheus
cAdvisor --> Prometheus
SystemMetrics --> Prometheus
DockerStats --> cAdvisor

%% Monitoring
Prometheus --> Grafana
```

Vue simplifiée (ASCII)

CLIENTS

 Web Frontend (Next.js)	 Mobile App (Flutter)
 API Gateway (Express)	 Admin Dashboard

MICROSERVICES BACKEND

 Auth Service	 Application Service	 Company Service
 Contact Service	 Interview Service	 Notification S.
 Dashboard Service	 Call Service	 Profile Service
 Event Service	 Followup Service	 Workflow Service
 Security Service	 System Metrics	 Deployment S.

INFRASTRUCTURE

 PostgreSQL 15	 Redis 7	 Prometheus
 Grafana	 Jaeger	 Docker
 Nginx Proxy	 SSL/TLS	 Logging

Composants principaux

1. **API Gateway** : Point d'entrée unique (Port 3000)
2. **Frontend Next.js** : Interface web moderne (Port 8080)
3. **Microservices** : 18+ services métier spécialisés
4. **Base de données** : PostgreSQL centralisée avec schémas séparés
5. **Cache** : Redis pour les sessions et performances
6. **Monitoring** : Prometheus + Grafana + cAdvisor
7. **Applications mobiles** : Flutter pour iOS/Android

API Gateway

Responsabilités

- **Routage** : Redirection des requêtes vers les bons services
- **Authentification** : Vérification des tokens JWT
- **Rate Limiting** : Limitation du nombre de requêtes

- **Logging** : Journalisation centralisée
- **Monitoring** : Collecte de métriques
- **CORS** : Gestion des origines autorisées
- **Load Balancing** : Répartition de charge entre services

Configuration

```
// Exemple de configuration du gateway
const services = {
  auth: 'http://auth-service:3001',
  applications: 'http://application-service:3002',
  companies: 'http://company-service:3003',
  contacts: 'http://contact-service:3004',
  interviews: 'http://interview-service:3005',
  calls: 'http://call-service:3006',
  events: 'http://event-service:3007',
  followups: 'http://followup-service:3008',
  profiles: 'http://profile-service:3009',
  notifications: 'http://notification-service:3010',
  workflows: 'http://workflow-service:3011',
  dashboard: 'http://dashboard-service:3012',
  security: 'http://security-service:3013',
  metrics: 'http://system-metrics-service:3018',
  deployment: 'http://deployment-service:3016',
  dockerstats: 'http://docker-stats-service:3015'
};

// Routage avec authentification
app.use('/api/auth', proxy(services.auth));
app.use('/api/applications', authenticate, proxy(services.applications));
app.use('/api/companies', authenticate, proxy(services.companies));

// Rate limiting
app.use('/api', rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 1000, // limite de 1000 requêtes par fenêtre
  message: 'Trop de requêtes, veuillez réessayer plus tard'
}));
```



Microservices



Service d'authentification (Auth Service)

Port : 3001 **Base de données** : PostgreSQL (schéma public) **Profil Docker** : auth

Responsabilités :

- Authentification des utilisateurs
- Gestion des tokens JWT
- Gestion des rôles et permissions
- Validation des sessions
- Intégration SMTP pour notifications
- Gestion des mots de passe (hachage bcrypt)

Endpoints principaux :

- POST /auth/login - Connexion
- POST /auth/register - Inscription
- POST /auth/refresh - Rafraîchissement token
- GET /auth/verify - Vérification token
- POST /auth/logout - Déconnexion
- POST /auth/forgot-password - Mot de passe oublié
- POST /auth/reset-password - Réinitialisation mot de passe

**Service des candidatures (Application Service)**

Port : 3002 **Base de données :** PostgreSQL (schéma public) **Profil Docker :** applications

Responsabilités :

- Gestion complète du cycle de vie des candidatures
- Statuts et workflows de candidatures
- Recherche et filtrage avancés
- Intégrations avec les entreprises
- Historique des changements de statut
- Relations many-to-many avec contacts

Endpoints principaux :

- GET /applications - Liste des candidatures
- POST /applications - Créer une candidature
- PUT /applications/:id - Mettre à jour
- DELETE /applications/:id - Supprimer
- GET /applications/:id/status - Statut d'une candidature
- GET /applications/:id/history - Historique des statuts

**Service des entreprises (Company Service)**

Port : 3003 **Base de données :** PostgreSQL (schéma public) **Profil Docker :** companies

Responsabilités :

- Gestion des entreprises et contacts
- Informations détaillées des entreprises

- Relations entre entreprises
- Historique des interactions
- Relations many-to-many avec contacts

Endpoints principaux :

- GET /companies - Liste des entreprises
- POST /companies - Créer une entreprise
- PUT /companies/:id - Mettre à jour
- DELETE /companies/:id - Supprimer
- GET /companies/:id/contacts - Contacts d'une entreprise
- GET /companies/search - Recherche d'entreprises

Service des contacts (Contact Service)

Port : 3004 **Base de données :** PostgreSQL (schéma public) **Profil Docker :** contacts

Responsabilités :

- Gestion des contacts et relations
- Historique des interactions
- Classification et tags
- Synchronisation avec les entreprises
- Relations many-to-many avec entreprises, candidatures, entretiens, événements

Endpoints principaux :

- GET /contacts - Liste des contacts
- POST /contacts - Créer un contact
- PUT /contacts/:id - Mettre à jour
- DELETE /contacts/:id - Supprimer
- GET /contacts/:id/history - Historique d'un contact
- GET /contacts/:id/companies - Entreprises du contact

Service des entretiens (Interview Service)

Port : 3005 **Base de données :** PostgreSQL (schéma public) **Profil Docker :** interviews

Responsabilités :

- Planification des entretiens
- Gestion des rendez-vous
- Notes et commentaires
- Suivi post-entretien
- Relations avec contacts et candidatures

Endpoints principaux :

- GET /interviews - Liste des entretiens

- `POST /interviews` - Planifier un entretien
- `PUT /interviews/:id` - Mettre à jour
- `DELETE /interviews/:id` - Annuler
- `POST /interviews/:id/notes` - Ajouter des notes
- `GET /interviews/upcoming` - Entretiens à venir

Service des appels (Call Service)

Port : 3006 **Base de données** : PostgreSQL (schéma public) **Profil Docker** : calls

Responsabilités :

- Gestion des appels téléphoniques
- Historique des communications
- Notes et comptes-rendus
- Rappels et planification
- Relations avec contacts

Endpoints principaux :

- `GET /calls` - Liste des appels
- `POST /calls` - Enregistrer un appel
- `PUT /calls/:id` - Mettre à jour
- `DELETE /calls/:id` - Supprimer
- `POST /calls/:id/notes` - Ajouter des notes
- `GET /calls/scheduled` - Appels planifiés



Service des événements (Event Service)

Port : 3007 **Base de données** : PostgreSQL (schéma public) **Profil Docker** : events

Responsabilités :

- Gestion des événements et calendrier
- Rappels automatiques
- Notifications d'événements
- Synchronisation calendrier
- Relations polymorphes avec tous les modules

Endpoints principaux :

- `GET /events` - Liste des événements
- `POST /events` - Créer un événement
- `PUT /events/:id` - Mettre à jour
- `DELETE /events/:id` - Supprimer
- `GET /events/upcoming` - Événements à venir
- `GET /events/calendar` - Vue calendrier

Service de suivi (Followup Service)

Port : 3008 **Base de données :** PostgreSQL (schéma public) **Profil Docker :** followups

Responsabilités :

- Suivi et relance des candidatures
- Workflows automatisés
- Planification des relances
- Historique des actions
- Relations avec contacts et candidatures

Endpoints principaux :

- GET /followups - Liste des suivis
- POST /followups - Créer un suivi
- PUT /followups/:id - Mettre à jour
- DELETE /followups/:id - Supprimer
- POST /followups/:id/complete - Marquer comme terminé
- GET /followups/due - Suivis à effectuer

Service des profils (Profile Service)

Port : 3009 **Base de données :** PostgreSQL (schéma public) **Profil Docker :** profiles

Responsabilités :

- Gestion des profils utilisateurs
- Préférences et paramètres
- Personnalisation de l'interface
- Gestion des avatars
- Paramètres de notification

Endpoints principaux :

- GET /profiles/me - Profil de l'utilisateur
- PUT /profiles/me - Mettre à jour le profil
- PUT /profiles/preferences - Préférences
- POST /profiles/avatar - Changer l'avatar
- GET /profiles/settings - Paramètres utilisateur

Service de notifications (Notification Service)

Port : 3010 **Base de données :** PostgreSQL (schéma public) **Profil Docker :** notifications

Responsabilités :

- Système de notifications multi-canaux
- Notifications par email (SMTP)
- Notifications push

- Historique des notifications
- Templates de notifications

Endpoints principaux :

- GET /notifications - Liste des notifications
- POST /notifications - Envoyer une notification
- PUT /notifications/:id/read - Marquer comme lue
- DELETE /notifications/:id - Supprimer
- GET /notifications/settings - Paramètres de notification
- POST /notifications/test - Test de notification

Service de workflows (Workflow Service)

Port : 3011 **Base de données :** PostgreSQL (schéma public) **Profil Docker :** workflows

Responsabilités :

- Définition et exécution de workflows
- Automatisation des processus
- Triggers et conditions
- Intégration avec d'autres services
- Workflows de relance automatique

Endpoints principaux :

- GET /workflows - Liste des workflows
- POST /workflows - Créer un workflow
- PUT /workflows/:id - Mettre à jour
- DELETE /workflows/:id - Supprimer
- POST /workflows/:id/execute - Exécuter un workflow
- GET /workflows/templates - Templates disponibles

Service de dashboard (Dashboard Service)

Port : 3012 **Base de données :** PostgreSQL (schéma public) **Profil Docker :** dashboard

Responsabilités :

- Analytics et métriques
- Tableaux de bord personnalisables
- Rapports et statistiques
- Export de données
- KPIs en temps réel

Endpoints principaux :

- GET /dashboard/overview - Vue d'ensemble
- GET /dashboard/analytics - Analytics détaillés

- GET /dashboard/reports - Rapports
- POST /dashboard/export - Exporter des données
- GET /dashboard/metrics - Métriques temps réel

Service de sécurité (Security Service)

Port : 3013 **Base de données :** PostgreSQL (schéma public) **Profil Docker :** security

Responsabilités :

- Gestion de la sécurité système
- Audit et logging de sécurité
- Détection d'intrusions
- Gestion des permissions avancées
- Analyse des menaces

Endpoints principaux :

- GET /security/audit - Logs d'audit
- POST /security/alert - Créer une alerte
- GET /security/threats - Menaces détectées
- PUT /security/settings - Paramètres de sécurité
- GET /security/sessions - Sessions actives

Service de métriques système (System Metrics Service)

Port : 3018 **Base de données :** PostgreSQL (schéma metrics) **Profil Docker :** full

Responsabilités :

- Collecte de métriques système avancées
- Analyse des performances
- Monitoring des ressources
- Alertes de performance
- Intégration Prometheus

Endpoints principaux :

- GET /system-metrics - Métriques système
- GET /system-metrics/history - Historique
- POST /system-metrics/alert - Configurer une alerte
- GET /system-metrics/performance - Analyse performance

Service de déploiement (Deployment Service)

Port : 3016 **Base de données :** PostgreSQL (schéma deployment) **Profil Docker :** full

Responsabilités :

- Gestion des déploiements
- Orchestration des services
- Rollback automatique
- Monitoring des déploiements
- CI/CD integration

Endpoints principaux :

- `GET /deployment/status` - Statut des déploiements
- `POST /deployment/deploy` - Lancer un déploiement
- `POST /deployment/rollback` - Rollback
- `GET /deployment/history` - Historique des déploiements



Service de statistiques Docker (Docker Stats Service)

Port : 3015 **Base de données :** PostgreSQL (schéma public) **Profil Docker :** full

Responsabilités :

- Collecte des statistiques Docker
- Monitoring des conteneurs
- Analyse d'utilisation des ressources
- Alertes de capacité
- Intégration cAdvisor

Endpoints principaux :

- `GET /docker-stats` - Statistiques Docker
- `GET /docker-stats/containers` - Conteneurs actifs
- `GET /docker-stats/resources` - Utilisation ressources
- `POST /docker-stats/alert` - Configurer une alerte



Base de données

Architecture des données

L'architecture utilise une base de données PostgreSQL centralisée avec des schémas séparés pour chaque service, garantissant :

- **Isolation logique** : Les données sont isolées par schéma
- **Scalabilité** : Chaque schéma peut être mis à l'échelle indépendamment
- **Sécurité** : Accès limité aux schémas par service
- **Maintenance** : Gestion centralisée plus simple

Configuration de la base de données

```
# docker-compose.yml
postgres:
  image: postgres:15-alpine
  environment:
    POSTGRES_DB: jobbingtrack
    POSTGRES_USER: jobbingtrack
    POSTGRES_PASSWORD: jobbingtrack123
  volumes:
    - postgres_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U jobbingtrack -d jobbingtrack"]
    interval: 10s
    timeout: 5s
    retries: 5
```



Structure étendue v4.1

La base de données a été étendue avec de nouveaux modèles et relations :

Modèles principaux ajoutés :

- **ApplicationStatusHistory** : Historique des changements de statut
- **Notification** : Système de notifications multi-canaux
- **Event** : Calendrier avec relations polymorphes
- **SyncQueue** : Synchronisation mobile/offline

Relations many-to-many :

- Contact ↔ Entreprise (ContactCompany)
- Contact ↔ Candidature (ContactApplication)
- Contact ↔ Relance (FollowUpContact)
- Contact ↔ Entretien (InterviewContact)
- Contact ↔ Événement (ContactEvent)

Fonctionnalités avancées :

- Relations polymorphes dans Event (un événement peut pointer vers une candidature, un entretien, une relance, ou un appel)
- Historisation complète des changements de statut
- Queue de synchronisation pour le mode offline
- Notifications avec métadonnées et entités liées

Schémas par service

Schéma d'authentification (auth)

```
-- Création du schéma
CREATE SCHEMA auth;

-- Table des utilisateurs
CREATE TABLE auth.users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(50) DEFAULT 'user',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table des sessions
CREATE TABLE auth.sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users(id),
  token VARCHAR(500) NOT NULL,
  expires_at TIMESTAMP NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Schéma des candidatures (applications)


```

-- Création du schéma
CREATE SCHEMA applications;

-- Table des candidatures
CREATE TABLE applications.applications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL,
    company_id UUID,
    position VARCHAR(255) NOT NULL,
    status VARCHAR(50) DEFAULT 'applied',
    description TEXT,
    salary_min DECIMAL,
    salary_max DECIMAL,
    location VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Historique des statuts
CREATE TABLE applications.application_status_history (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    application_id UUID REFERENCES applications.applications(id),
    old_status VARCHAR(50),
    new_status VARCHAR(50) NOT NULL,
    changed_by UUID,
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Schéma des entreprises (companies)

```

-- Création du schéma
CREATE SCHEMA companies;

-- Table des entreprises
CREATE TABLE companies.companies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  description TEXT,
  website VARCHAR(255),
  contact_email VARCHAR(255),
  industry VARCHAR(100),
  size VARCHAR(50),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table des contacts
CREATE TABLE companies.contacts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID REFERENCES companies.companies(id),
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255),
  phone VARCHAR(50),
  position VARCHAR(100),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table de liaison many-to-many Contact-Entreprise
CREATE TABLE companies.contact_companies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  contact_id UUID NOT NULL,
  company_id UUID NOT NULL REFERENCES companies.companies(id),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(contact_id, company_id)
);

```

Monitoring

Stack de monitoring complet

- **Prometheus** (Port 9090) : Collecte et stockage des métriques
- **Grafana** (Port 3001) : Visualisation des métriques et dashboards
- **Metrics Aggregator** (Port 3014) : Agrégation et centralisation
- **cAdvisor** (Port 8081) : Métriques des conteneurs Docker
- **System Metrics Service** (Port 3018) : Métriques système avancées

- **Docker Stats Service** (Port 3015) : Statistiques Docker temps réel
- **Jaeger** : Tracing distribué (optionnel)

Services de monitoring



Metrics Aggregator Service

Responsabilités :

- Collecte centralisée des métriques
- Agrégation des données
- Interface web de monitoring
- Export vers Prometheus

Configuration :

```
# docker-compose.yml
jobbingtrack-metrics-aggregator:
  image: jobbingtrack-metrics-aggregator
  ports:
    - "3014:3014"
  environment:
    - CADVISOR_URL=http://cadvisor:8080
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock:ro
```



cAdvisor

Responsabilités :

- Monitoring des conteneurs Docker
- Métriques CPU, mémoire, disque
- Statistiques réseau et I/O
- Interface web intégrée

Configuration :

```
cadvisor:
  image: gcr.io/cadvisor/cadvisor:v0.47.2
  privileged: true
  ports:
    - "8081:8080"
  volumes:
    - /:/rootfs:ro
    - /var/run:/var/run:ro
    - /sys:/sys:ro
    - /var/lib/docker:/var/lib/docker:ro
```

System Metrics Service

Responsabilités :

- Collecte de métriques système avancées
- Analyse des performances
- Base de données dédiée (schéma metrics)
- Alertes configurables

Métriques collectées

Métriques système

- **CPU** : Utilisation par cœur, charge système
- **Mémoire** : RAM utilisée, swap, cache
- **Disque** : Espace utilisé, I/O, latence
- **Réseau** : Trafic entrant/sortant, connexions

Métriques applicatives

- **Requêtes** : Nombre par endpoint, latence moyenne
- **Erreurs** : Taux d'erreur HTTP, exceptions
- **Base de données** : Connexions actives, requêtes lentes
- **Cache** : Taux de succès Redis, évictions

Métriques Docker

- **Conteneurs** : CPU, mémoire par conteneur
- **Images** : Taille, nombre d'images
- **Volumes** : Utilisation des volumes
- **Réseau** : Trafic inter-conteneurs

Configuration Prometheus

```
# monitoring/prometheus/prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'api-gateway'
    static_configs:
      - targets: ['api-gateway:3000']
    metrics_path: '/metrics'

  - job_name: 'auth-service'
    static_configs:
      - targets: ['auth-service:3001']
    metrics_path: '/metrics'

  - job_name: 'application-service'
    static_configs:
      - targets: ['application-service:3002']
    metrics_path: '/metrics'

  - job_name: 'metrics-aggregator'
    static_configs:
      - targets: ['jobbingtrack-metrics-aggregator:3014']
    metrics_path: '/metrics'

  - job_name: 'cadvisor'
    static_configs:
      - targets: ['cadvisor:8080']
    metrics_path: '/metrics'
```

Sécurité

Service de sécurité (Security Service)

Port : 3013 **Base de données :** PostgreSQL (schéma security)

Responsabilités :

- Audit et logging de sécurité
- Détection d'intrusions
- Gestion des permissions avancées
- Analyse des menaces
- Gestion des sessions utilisateur

Fonctionnalités :

- Logs d'audit centralisés
- Alertes de sécurité temps réel
- Gestion des sessions utilisateur
- Protection contre les attaques
- Analyse des patterns suspects

Authentification et autorisation

JWT Authentication

- **Tokens JWT** : Authentification stateless
- **Refresh Tokens** : Renouvellement automatique
- **Expiration configurable** : Sécurité renforcée
- **Multi-device support** : Gestion des sessions

RBAC (Role-Based Access Control)

```
// Configuration des rôles
const roles = {
  admin: ['read', 'write', 'delete', 'manage_users', 'system_admin'],
  manager: ['read', 'write', 'manage_team', 'reports'],
  user: ['read', 'write_own', 'basic_profile'],
  guest: ['read']
};

// Middleware d'authentification
const authenticate = async (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'Token manquant' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (error) {
    res.status(401).json({ error: 'Token invalide' });
  }
};
```

Rate Limiting

```
// Configuration du rate limiting
const rateLimit = {
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // limite de 1000 requêtes par fenêtre
  message: 'Trop de requêtes, veuillez réessayer plus tard'
};

// Par endpoint
const authRateLimit = rateLimit({
  windowMs: 5 * 60 * 1000, // 5 minutes pour auth
  max: 5, // 5 tentatives de connexion
  skipSuccessfulRequests: true
});
```

Chiffrement et secrets

Variables d'environnement sécurisées

```
# docker-compose.yml
auth-service:
  environment:
    - JWT_SECRET=${JWT_SECRET:-your-secret-key-change-in-production-2025}
    - JWT_REFRESH_SECRET=${JWT_REFRESH_SECRET:-your-refresh-secret-change-too-2025}
    - SMTP_PASS=${SMTP_PASS:-V**Uw61^3*bz5c2AFrx&2d&%}
    - DATABASE_PASSWORD=${DATABASE_PASSWORD:-secure-db-password-2025}
```

Hachage des mots de passe

- **Bcrypt** : Hachage sécurisé avec salt
- **Work factor** : 12 rounds minimum
- **Salt unique** : Par utilisateur

Communication sécurisée

- **HTTPS** : TLS 1.3 obligatoire en production
- **CORS** : Configuration restrictive
- **Headers de sécurité** : HSTS, CSP, X-Frame-Options, X-Content-Type-Options

Audit et monitoring de sécurité

Logs d'audit

```
-- Table d'audit de sécurité
CREATE TABLE security.audit_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID,
  action VARCHAR(100) NOT NULL,
  resource VARCHAR(255),
  resource_id UUID,
  ip_address INET,
  user_agent TEXT,
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  details JSONB
);
```

Alertes de sécurité automatiques

- Tentatives de connexion échouées multiples
- Accès non autorisés aux endpoints sensibles
- Actions suspectes (pattern matching)
- Changements de configuration système
- Tentatives d'injection SQL/XSS

Déploiement

Architecture de déploiement


```

graph TB
    subgraph "Load Balancer / Reverse Proxy"
        LB[Traefik / Nginx<br/>Port: 80, 443]
    end

    subgraph "Services essentiels (toujours actifs)"
        Postgres[(🗄️ PostgreSQL<br/>Port: 5432)]
        Redis[(📦 Redis<br/>Port: 6379)]
        Gateway[🏠 API Gateway<br/>Port: 3000]
        Frontend[🌐 Frontend Next.js<br/>Port: 8080]
    end

    subgraph "Services principaux (profil full)"
        Auth1[🔑 Auth Service<br/>Port: 3001]
        Auth2[🔑 Auth Service<br/>Port: 3001]
        App1[📄 Application Service<br/>Port: 3002]
        App2[📄 Application Service<br/>Port: 3002]
        Company[🏢 Company Service<br/>Port: 3003]
        Contact[👥 Contact Service<br/>Port: 3004]
        Interview[🎤 Interview Service<br/>Port: 3005]
        Call[📞 Call Service<br/>Port: 3006]
        Event[📅 Event Service<br/>Port: 3007]
        Followup[🔄 Followup Service<br/>Port: 3008]
        Profile[👤 Profile Service<br/>Port: 3009]
        Notification[🔔 Notification Service<br/>Port: 3010]
        Workflow[⚙️ Workflow Service<br/>Port: 3011]
    end

    subgraph "Services système (profil full)"
        Security[🔒 Security Service<br/>Port: 3013]
        SystemMetrics[📊 System Metrics<br/>Port: 3018]
        Deployment[🚀 Deployment Service<br/>Port: 3016]
        DockerStats[🐳 Docker Stats<br/>Port: 3015]
    end

    subgraph "Monitoring (profil monitoring)"
        MetricsAgg[📊 Metrics Aggregator<br/>Port: 3014]
        cAdvisor[🖥️ cAdvisor<br/>Port: 8081]
        Prometheus[📊 Prometheus<br/>Port: 9090]
        Grafana[📊 Grafana<br/>Port: 3001]
    end

    %% Connexions
    LB --> Gateway
    LB --> Frontend

    Gateway --> Auth1
    Gateway --> Auth2
    Gateway --> App1
    Gateway --> App2

```

```

Gateway --> Company
Gateway --> Contact
Gateway --> Interview
Gateway --> Call
Gateway --> Event
Gateway --> Followup
Gateway --> Profile
Gateway --> Notification
Gateway --> Workflow
Gateway --> Security
Gateway --> SystemMetrics
Gateway --> Deployment
Gateway --> DockerStats

```

```
%% Bases de données
```

```

Auth1 --> Postgres
Auth2 --> Postgres
App1 --> Postgres
App2 --> Postgres
Company --> Postgres
Contact --> Postgres
Interview --> Postgres
Call --> Postgres
Event --> Postgres
Followup --> Postgres
Profile --> Postgres
Notification --> Postgres
Workflow --> Postgres
Security --> Postgres
SystemMetrics --> Postgres
Deployment --> Postgres

```

```
%% Cache et monitoring
```

```

Auth1 --> Redis
Auth2 --> Redis
Notification --> Redis
MetricsAgg --> cAdvisor
MetricsAgg --> Prometheus
cAdvisor --> Prometheus
SystemMetrics --> Prometheus
DockerStats --> cAdvisor

```

```
%% Monitoring
```

```
Prometheus --> Grafana
```

Profils de déploiement

Services essentiels (toujours démarrés)

```
make up # Services de base (postgres, redis, api-gateway, frontend)
```

Services par fonctionnalités

```
make up-profile PROFILE=auth          # Authentification
make up-profile PROFILE=applications # Candidatures
make up-profile PROFILE=companies     # Entreprises
make up-profile PROFILE=contacts      # Contacts
make up-profile PROFILE=interviews    # Entretiens
make up-profile PROFILE=calls         # Appels
make up-profile PROFILE=events        # Événements
make up-profile PROFILE=followups     # Suivis
make up-profile PROFILE=profiles      # Profils
make up-profile PROFILE=notifications # Notifications
make up-profile PROFILE=workflows     # Workflows
```

Services système et monitoring

```
make up-profile PROFILE=dashboard # Dashboard
make up-profile PROFILE=security  # Sécurité
make up-profile PROFILE=monitoring # Monitoring complet
make up-profile PROFILE=full      # Tous les services
```

Configuration Docker

Dockerfile standard pour les services

```
# Exemple Dockerfile pour un service Node.js
FROM node:20-alpine

WORKDIR /app

# Copier les fichiers de dépendances
COPY package*.json ./
RUN npm ci --only=production

# Copier le code source
COPY src/ ./src/
COPY prisma/ ./prisma/

# Variables d'environnement
ENV NODE_ENV=production
ENV PORT=3000

# Exposer le port
EXPOSE 3000

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD node healthcheck.js

# Commande de démarrage
CMD ["node", "src/server.js"]
```

Dockerfile pour le frontend

```
# frontend/Dockerfile
FROM node:20-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

# Production image
FROM node:20-alpine AS runner

WORKDIR /app
COPY --from=builder /app/public ./public
COPY --from=builder /app/.next ./next
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./package.json

EXPOSE 3000
CMD ["npm", "start"]
```

Variables d'environnement

```
# .env.example
POSTGRES_DB=jobbingtrack
POSTGRES_USER=jobbingtrack
POSTGRES_PASSWORD=jobbingtrack123

JWT_SECRET=your-secret-key-change-in-production-2025
JWT_REFRESH_SECRET=your-refresh-secret-change-too-2025

REDIS_URL=redis://redis:6379

SMTP_HOST=smtp.ovh.net
SMTP_PORT=587
SMTP_USER=candidatures@delhomme.ovh
SMTP_PASS=your-smtp-password
SMTP_FROM=JobbingTrack <candidatures@delhomme.ovh>

ALLOWED_ORIGINS=https://jobbingtrack.com,https://app.jobbingtrack.com
FRONTEND_URL=https://jobbingtrack.com

LOG_LEVEL=info
NODE_ENV=production
```



Applications mobiles

Flutter Mobile App

Port : 8090 (interface web de l'émulateur) **Base de données** : SQLite locale + synchronisation PostgreSQL

Framework : Flutter (Dart)

Fonctionnalités :

- Interface mobile native iOS/Android
- Synchronisation en temps réel
- Mode hors ligne
- Notifications push
- Scanner de cartes de visite
- Widget de notification mobile

Architecture :

```
// Structure de l'app Flutter
lib/
├─ models/           # Modèles de données
├─ services/         # Services API et locaux
├─ providers/        # State management (Riverpod)
├─ screens/          # Écrans de l'application
├─ widgets/          # Composants réutilisables
└─ utils/            # Utilitaires
```

Configuration Docker :

```
# flutter-mobile-app/Dockerfile
FROM cirrusci/flutter:stable

WORKDIR /app
COPY pubspec.yaml pubspec.lock ./
RUN flutter pub get

COPY . .
RUN flutter build web

EXPOSE 8080
CMD ["flutter", "run", "-d", "web-server", "--web-port", "8080"]
```



Communication inter-services

Communication synchrone

HTTP/REST via API Gateway

```
// Exemple de requête du frontend vers un service
const response = await fetch('/api/applications', {
  method: 'GET',
  headers: {
    'Authorization': `Bearer ${token}`,
    'Content-Type': 'application/json',
  },
});

// API Gateway route vers le service approprié
app.use('/api/applications', authenticate, proxy('http://application-service:3002'
```

Service-to-Service Communication

```
// Communication directe entre services
const authService = await fetch('http://auth-service:3001/verify', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ token }),
});
```

Communication asynchrone

Événements et workflows

```
// Publication d'un événement
const event = {
  type: 'application.created',
  data: { applicationId, userId, companyId },
  timestamp: new Date().toISOString(),
};

// Traitement des événements dans le workflow service
workflowService.processEvent(event);
```

Notifications

```
// Envoi de notification asynchrone
notificationService.send({
  type: 'email',
  to: user.email,
  template: 'application-status-changed',
  data: { application, newStatus },
});
```

Protocoles de communication

REST APIs

- **JSON** : Format standard
- **Pagination** : Limit/offset et cursor-based
- **Filtering** : Query parameters
- **Sorting** : Ordre des résultats

WebSockets (temps réel)

```
// Connexions WebSocket pour les notifications
const ws = new WebSocket('ws://localhost:3000/notifications');
ws.onmessage = (event) => {
  const notification = JSON.parse(event.data);
  updateUI(notification);
};
```



Scalabilité et performance

Scaling horizontal

Load Balancing avec Traefik


```
# Configuration Traefik pour API Gateway
services:
  api-gateway:
    deploy:
      replicas: 3
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.api.rule=Host(`api.jobbingtrack.local`)"
      - "traefik.http.services.api.loadbalancer.server.port=3000"
```

Auto-scaling avec Docker Swarm

```
# Créer un service scalable
docker service create \
  --name jobbingtrack-auth \
  --replicas 3 \
  --network jobbingtrack-network \
  jobbingtrack-auth-service
```

Database Connection Pooling

```
// Configuration du pool de connexions PostgreSQL
const poolConfig = {
  max: 20,          // nombre maximum de connexions
  min: 5,           // nombre minimum de connexions
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
};
```

Optimisations de performance

Caching multi-niveaux

```
// Cache Redis pour les sessions
const redisConfig = {
  host: 'redis',
  port: 6379,
  ttl: 3600, // 1 heure
};

// Cache en mémoire pour les métadonnées
const memoryCache = new Map();
```

Database Indexing

```
-- Index sur les tables fréquemment consultées
CREATE INDEX idx_applications_user_id ON applications.applications(user_id);
CREATE INDEX idx_applications_status ON applications.applications(status);
CREATE INDEX idx_companies_name ON companies.companies(name);
CREATE INDEX idx_contacts_company_id ON companies.contacts(company_id);
CREATE INDEX idx_application_status_history_application_id ON applications.applica
```

CDN et assets optimisés

```
// Configuration Next.js pour les assets statiques
const nextConfig = {
  images: {
    domains: ['cdn.jobbingtrack.com'],
    formats: ['image/webp', 'image/avif'],
  },
  experimental: {
    optimizeCss: true,
    webVitalsAttribution: ['CLS', 'LCP'],
  },
};
```



Patterns et bonnes pratiques

API Design

- **RESTful** avec conventions standard
- **Versioning** des endpoints (/api/v1/)
- **Pagination** pour les listes volumineuses

- **Filtering et sorting** cohérents
- **Validation** avec Zod
- **Documentation** OpenAPI/Swagger

Gestion d'Erreurs

- **Codes d'erreur** standardisés
- **Messages d'erreur** localisés
- **Logging structuré** pour le debugging
- **Graceful degradation** en cas d'erreur
- **Circuit breaker** pattern

Code Quality

- **ESLint + Prettier** pour la cohérence
- **TypeScript strict** partout
- **Tests obligatoires** pour les nouvelles features
- **Documentation** des APIs avec Swagger/OpenAPI
- **SOLID principes** appliqués

Logging et tracing

Logging centralisé

```
// Configuration Winston pour tous les services
const logger = winston.createLogger({
  level: 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  defaultMeta: { service: 'auth-service' },
  transports: [
    new winston.transports.File({ filename: 'error.log', level: 'error' }),
    new winston.transports.File({ filename: 'combined.log' }),
  ],
});
```

Distributed Tracing

```
// Intégration OpenTelemetry
const { trace } = require('@opentelemetry/api');

const tracer = trace.getTracer('auth-service');

app.use('/login', async (req, res) => {
  const span = tracer.startSpan('user-login');
  try {
    // logique de connexion
    span.setStatus({ code: 1 }); // OK
  } catch (error) {
    span.setStatus({ code: 2, message: error.message }); // ERROR
  } finally {
    span.end();
  }
});
```

Ressources supplémentaires

Documentation technique

- [Guide de développement](#) - Environnement de développement
- [Documentation API](#) - Endpoints et spécifications
- [Guide de déploiement](#) - Déploiement en production
- [Documentation Makefile](#) - Commandes et automatisations
- [Guide des tests](#) - Stratégies de tests

Guides spécialisés

- [Guide de sécurité](#) - Bonnes pratiques sécurité
- [Guide des performances](#) - Optimisations
- [Guide de monitoring](#) - Surveillance système
- [Guide Portainer](#) - Interface de gestion Docker

Outils et utilitaires

- [Scripts d'administration](#) - Automatisations
 - [Configuration des tests](#) - Suite de tests complète
 - [Monitoring et métriques](#) - Dashboards Prometheus/Grafana
-

Navigation

Document	Description
 README Principal	Vue d'ensemble du projet
 Index Documentation	Liste de tous les guides
 Services	Détail des microservices
 API	Documentation des APIs
 Tests	Stratégies de test
 Dépannage	Résolution des problèmes

Dernière mise à jour: Octobre 2025 **Version:** 4.1 - Architecture microservices complète et documentée

Équipe: JobbingTrack Development Team



Services Microservices - JobbingTrack

Source: `core/services.md`

Documentation détaillée des 18+ microservices de JobbingTrack v4.1.

[← Retour au README principal](#)



Vue d'ensemble

JobbingTrack utilise une architecture de microservices moderne organisée autour de **services essentiels** (toujours démarrés) et **services optionnels** (démarrés selon les besoins via des profils Docker Compose).



Services par catégories



Services essentiels (toujours actifs)

Service	Port	Description	Docker Compose
PostgreSQL	5432	Base de données principale	<code>make up</code>
Redis	6379	Cache et sessions	<code>make up</code>
API Gateway	3000	Point d'entrée unique	<code>make up</code>
Frontend	8080	Interface Next.js	<code>make up</code>
Metrics Aggregator	3014	Monitoring centralisé	<code>make up-profile monitoring</code>
cAdvisor	8081	Monitoring Docker	<code>make up-profile monitoring</code>



Services métier principaux

Service	Port	Profil	Description
Auth Service	3001	<code>auth</code>	Authentification JWT, utilisateurs
Application Service	3002	<code>applications</code>	Gestion des candidatures
Company Service	3003	<code>companies</code>	Gestion des entreprises
Contact Service	3004	<code>contacts</code>	Gestion des contacts
Interview Service	3005	<code>interviews</code>	Planification entretiens
Call Service	3006	<code>calls</code>	Gestion des appels

 Event Service	3007	events	Calendrier et événements
 Followup Service	3008	followups	Suivi et relances
 Profile Service	3009	profiles	Profils utilisateurs
 Notification Service	3010	notifications	Notifications multi-canaux
 Workflow Service	3011	workflows	Automatisation workflows
 Dashboard Service	3012	dashboard	Analytics et KPIs

Services système avancés

Service	Port	Profil	Description
 Security Service	3013	security	Audit et sécurité
 System Metrics	3018	full	Métriques système
 Deployment Service	3016	full	Gestion déploiements
 Docker Stats	3015	full	Statistiques Docker

Commandes de démarrage

Services essentiels (base)

```
make up # Démarre les services de base
```

Services par fonctionnalités

```
make up-profile PROFILE=auth          # Authentification
make up-profile PROFILE=applications  # Candidatures
make up-profile PROFILE=companies     # Entreprises
make up-profile PROFILE=contacts      # Contacts
make up-profile PROFILE=interviews    # Entretiens
make up-profile PROFILE=calls         # Appels
make up-profile PROFILE=events        # Événements
make up-profile PROFILE=followups     # Suivis
make up-profile PROFILE=profiles      # Profils
make up-profile PROFILE=notifications # Notifications
make up-profile PROFILE=workflows     # Workflows
```

Services système

```
make up-profile PROFILE=dashboard    # Dashboard
make up-profile PROFILE=security      # Sécurité
make up-profile PROFILE=monitoring    # Monitoring complet
make up-profile PROFILE=full         # Tous les services
```

Configuration réseau

Réseau Docker

```
# docker-compose.yml
networks:
  jobbingtrack-network:
    driver: bridge
    ipam:
      config:
        - subnet: 172.20.0.0/16
```

Variables d'environnement partagées


```
# .env
POSTGRES_DB=jobbingtrack
POSTGRES_USER=jobbingtrack
POSTGRES_PASSWORD=jobbingtrack123

JWT_SECRET=your-secret-key-change-in-production-2025
JWT_REFRESH_SECRET=your-refresh-secret-change-too-2025

REDIS_URL=redis://redis:6379

ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8080
FRONTEND_URL=http://localhost:8080
```



Monitoring et métriques

Services de monitoring

- **Prometheus** (Port 9090) - Collecte des métriques
- **Grafana** (Port 3001) - Visualisation
- **cAdvisor** (Port 8081) - Métriques Docker
- **Metrics Aggregator** (Port 3014) - Agrégation

Points d'accès monitoring

- **API Gateway Health** : `http://localhost:3000/health`
- **cAdvisor Web UI** : `http://localhost:8081`
- **Prometheus** : `http://localhost:9090`
- **Grafana** : `http://localhost:3001`



Détails techniques

Configuration Docker standard

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY src/ ./src/
COPY prisma/ ./prisma/
ENV NODE_ENV=production
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD node healthcheck.js
CMD ["node", "src/server.js"]
```

Health checks

Tous les services incluent des health checks automatiques :

- **HTTP** : Endpoint `/health` pour les services web
- **Database** : `pg_isready` pour PostgreSQL
- **Cache** : `redis-cli ping` pour Redis
- **Custom** : Scripts spécifiques par service

Logs et debugging

- **Logs centralisés** : Winston pour tous les services
- **Format structuré** : JSON avec métadonnées
- **Niveaux** : info, warn, error, debug
- **Rotation** : Automatique avec compression



Ressources

- [Architecture complète](#) - Vue technique détaillée
- [Base de données](#) - Schémas et relations
- [API Reference](#) - Documentation des endpoints
- [Déploiement](#) - Guide production

Version: 4.1 - Services microservices **Dernière mise à jour:** Octobre 2025



Base de Données - JobbingTrack

Source: *core/database.md*

Structure complète de la base de données PostgreSQL de JobbingTrack v4.1.

[← Retour au README principal](#)



Vue d'ensemble

Base de données PostgreSQL 15+ avec architecture multi-schémas, relations polymorphes et support mobile/offline.



Architecture

Configuration PostgreSQL

```
# docker-compose.yml
postgres:
  image: postgres:15-alpine
  environment:
    POSTGRES_DB: jobbingtrack
    POSTGRES_USER: jobbingtrack
    POSTGRES_PASSWORD: jobbingtrack123
  volumes:
    - postgres_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U jobbingtrack -d jobbingtrack"]
    interval: 10s
    timeout: 5s
    retries: 5
```

Principes de conception

- **Multi-schémas** : Isolation par service
 - **Relations polymorphes** : Flexibilité des associations
 - **Historisation** : Suivi des changements
 - **Synchronisation** : Support offline mobile
 - **Audit trail** : Traçabilité complète
-



Modèles principaux



Nouveautés v4.1

ApplicationStatusHistory

Historique des changements de statut des candidatures

```
CREATE TABLE applications.application_status_history (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  application_id UUID REFERENCES applications(applications(id),
  old_status VARCHAR(50),
  new_status VARCHAR(50) NOT NULL,
  changed_by UUID,
  changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  notes TEXT
);
```

Notification

Système de notifications multi-canaux

```
CREATE TABLE notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL,
  type VARCHAR(50) NOT NULL, -- email, push, sms, in_app
  title VARCHAR(255) NOT NULL,
  message TEXT NOT NULL,
  entity_type VARCHAR(50), -- application, interview, etc.
  entity_id UUID,
  is_read BOOLEAN DEFAULT FALSE,
  read_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Event

Calendrier avec relations polymorphes

```
CREATE TABLE events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL,
  title VARCHAR(255) NOT NULL,
  description TEXT,
  start_date TIMESTAMP NOT NULL,
  end_date TIMESTAMP,
  event_type VARCHAR(50) NOT NULL,
  entity_type VARCHAR(50), -- application, interview, call, followup
  entity_id UUID,
  reminder_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

SyncQueue

Synchronisation mobile/offline

```
CREATE TABLE sync_queues (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL,
  entity_type VARCHAR(50) NOT NULL,
  entity_id UUID NOT NULL,
  action VARCHAR(20) NOT NULL, -- create, update, delete
  data JSONB NOT NULL,
  synced BOOLEAN DEFAULT FALSE,
  synced_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```



Relations many-to-many

Contact ↔ Entreprise

```
CREATE TABLE companies.contact_companies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  contact_id UUID NOT NULL,
  company_id UUID NOT NULL REFERENCES companies.companies(id),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(contact_id, company_id)
);
```

Contact ↔ Candidature

```
CREATE TABLE applications.contact_applications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  contact_id UUID NOT NULL,
  application_id UUID NOT NULL REFERENCES applications.applications(id),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(contact_id, application_id)
);
```

Contact ↔ Relance

```
CREATE TABLE followups.follow_up_contacts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  contact_id UUID NOT NULL,
  follow_up_id UUID NOT NULL REFERENCES followups.followups(id),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(contact_id, follow_up_id)
);
```

Contact ↔ Entretien

```
CREATE TABLE interviews.interview_contacts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  contact_id UUID NOT NULL,
  interview_id UUID NOT NULL REFERENCES interviews.interviews(id),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(contact_id, interview_id)
);
```

Contact ↔ Événement

```
CREATE TABLE events.contact_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  contact_id UUID NOT NULL,
  event_id UUID NOT NULL REFERENCES events(id),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(contact_id, event_id)
);
```

Enums et types

Statuts de candidatures

```
CREATE TYPE application_status AS ENUM (
  'applied', 'screening', 'phone_interview', 'technical_interview',
  'final_interview', 'offer', 'rejected', 'withdrawn', 'archived'
);
```

Types d'événements

```
CREATE TYPE event_type AS ENUM (
  'interview', 'call', 'followup', 'deadline', 'reminder', 'meeting'
);
```

Types de notifications

```
CREATE TYPE notification_type AS ENUM (
  'email', 'push', 'sms', 'in_app'
);
```

Rôles utilisateurs

```
CREATE TYPE user_role AS ENUM (
    'user', 'admin', 'super_admin'
);
```

Indexes et performances

Indexes principaux

```
-- Applications
CREATE INDEX idx_applications_user_id ON applications.applications(user_id);
CREATE INDEX idx_applications_status ON applications.applications(status);
CREATE INDEX idx_applications_company_id ON applications.applications(company_id);
CREATE INDEX idx_applications_created_at ON applications.applications(created_at DESC);

-- Contacts
CREATE INDEX idx_contacts_user_id ON companies.contacts(user_id);
CREATE INDEX idx_contacts_company_id ON companies.contacts(company_id);
CREATE INDEX idx_contacts_email ON companies.contacts(email);
CREATE INDEX idx_contacts_last_contact ON companies.contacts(last_contact_date DESC);

-- Entretiens
CREATE INDEX idx_interviews_user_id ON interviews.interviews(user_id);
CREATE INDEX idx_interviews_date ON interviews.interviews(scheduled_at);
CREATE INDEX idx_interviews_status ON interviews.interviews(status);

-- Historique
CREATE INDEX idx_application_status_history_application_id ON applications.application_status_history(application_id);
CREATE INDEX idx_application_status_history_changed_at ON applications.application_status_history(changed_at);
```

Indexes many-to-many

```
-- Tables de liaison
CREATE INDEX idx_contact_companies_contact_id ON companies.contact_companies(contact_id);
CREATE INDEX idx_contact_companies_company_id ON companies.contact_companies(company_id);
CREATE INDEX idx_contact_applications_contact_id ON applications.contact_applications(contact_id);
CREATE INDEX idx_contact_applications_application_id ON applications.contact_applications(application_id);
```

Optimisations avancées


```
-- Index partiels (pour les données actives)
CREATE INDEX idx_applications_active ON applications.applications(user_id, status)
    WHERE is_archived = FALSE;

-- Index composites
CREATE INDEX idx_applications_user_status ON applications.applications(user_id, status);
CREATE INDEX idx_contacts_user_company ON companies.contacts(user_id, company_id);

-- Index GIN pour recherche textuelle
CREATE INDEX idx_companies_search ON companies.companies USING GIN (to_tsvector('fr', content));
```



Schémas par service

Auth (authentification)

```

-- Schéma: auth
CREATE SCHEMA auth;

-- Utilisateurs
CREATE TABLE auth.users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role user_role DEFAULT 'user',
  first_name VARCHAR(100),
  last_name VARCHAR(100),
  phone VARCHAR(50),
  profile_picture VARCHAR(500),
  is_active BOOLEAN DEFAULT TRUE,
  reset_token VARCHAR(255),
  reset_token_expiry TIMESTAMP,
  is_deleted BOOLEAN DEFAULT FALSE,
  is_archived BOOLEAN DEFAULT FALSE,
  sync_hash VARCHAR(64),
  entity_hash VARCHAR(64),
  last_sync_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Sessions
CREATE TABLE auth.sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users(id),
  token VARCHAR(500) NOT NULL,
  refresh_token VARCHAR(500),
  expires_at TIMESTAMP NOT NULL,
  ip_address INET,
  user_agent TEXT,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Applications (candidatures)

```

-- Schéma: applications
CREATE SCHEMA applications;

-- Candidatures
CREATE TABLE applications.applications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL,
  company_id UUID,
  platform_id UUID,
  position VARCHAR(255) NOT NULL,
  description TEXT,
  location VARCHAR(255),
  type VARCHAR(100),
  salary_min DECIMAL(10,2),
  salary_max DECIMAL(10,2),
  status application_status DEFAULT 'applied',
  application_date DATE,
  job_url VARCHAR(500),
  notes TEXT,
  is_archived BOOLEAN DEFAULT FALSE,
  archived_at TIMESTAMP,
  archived_by UUID,
  archived_reason TEXT,
  sync_hash VARCHAR(64),
  entity_hash VARCHAR(64),
  last_sync_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Companies (entreprises)

```
-- Schéma: companies
CREATE SCHEMA companies;

-- Entreprises
CREATE TABLE companies.companies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  website VARCHAR(255),
  industry VARCHAR(100),
  size VARCHAR(50),
  location VARCHAR(255),
  description TEXT,
  logo_url VARCHAR(500),
  sync_hash VARCHAR(64),
  entity_hash VARCHAR(64),
  last_sync_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Migration depuis v4.0

Étapes de migration

1. **Sauvegarde** de la base existante
2. **Application des migrations** : `npx prisma migrate dev`
3. **Mise à jour des services** pour utiliser les nouvelles relations
4. **Test des fonctionnalités** ajoutées

Script de migration

```
-- Migration des données existantes
-- Ajout des nouvelles colonnes et relations
-- Mise à jour des indexes
-- Validation de l'intégrité des données
```

Ressources

- [Architecture complète](#) - Vue technique
- [Services](#) - Détail des microservices

- [API Reference](#) - Documentation API
- [Prisma Schema](#) - Schéma source

Version: 4.1 - Base de données étendue **Dernière mise à jour:** Octobre 2025

API Reference - JobbingTrack

Source: *api/api-reference.md*

Documentation complète des APIs REST de JobbingTrack v4.1.

[← Retour au README principal](#)

Vue d'ensemble

API REST moderne avec architecture microservices, authentification JWT et documentation OpenAPI.

Configuration de base

Base URL

Production: `https://api.jobbingtrack.com`
Développement: `http://localhost:3000`

Headers requis

Content-Type: `application/json`
Authorization: `Bearer <jwt_token>`

Authentification

```
# Obtenir un token
curl -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"user@example.com","password":"password123"}'
```

Services API

Authentification

Base URL : /auth **Port direct :** 3001

Connexion

```
POST /auth/login
Content-Type: application/json

{
  "email": "user@example.com",
  "password": "password123"
}
```

Inscription

```
POST /auth/register
Content-Type: application/json

{
  "email": "newuser@example.com",
  "password": "password123",
  "firstName": "John",
  "lastName": "Doe"
}
```

Vérification token

```
GET /auth/verify
Authorization: Bearer <token>
```

Candidatures

Base URL : /applications **Port direct :** 3002

Lister les candidatures

```
GET /applications?status=applied&limit=20&offset=0
Authorization: Bearer <token>
```

Créer une candidature

```
POST /applications
Authorization: Bearer <token>
Content-Type: application/json

{
  "companyId": "uuid",
  "position": "Développeur Full Stack",
  "description": "Mission passionnante...",
  "location": "Paris",
  "salaryMin": 45000,
  "salaryMax": 55000,
  "status": "applied"
}
```

Mettre à jour une candidature

```
PUT /applications/{id}
Authorization: Bearer <token>
Content-Type: application/json

{
  "status": "interview",
  "notes": "Entretien planifié"
}
```

Entreprises

Base URL : /companies **Port direct :** 3003

Lister les entreprises

```
GET /companies?industry=tech&size=50-200
Authorization: Bearer <token>
```

Créer une entreprise


```
POST /companies
Authorization: Bearer <token>
Content-Type: application/json

{
  "name": "TechCorp",
  "website": "https://techcorp.com",
  "industry": "Technology",
  "size": "50-200",
  "description": "Entreprise innovante"
}
```

Contacts

Base URL : /contacts **Port direct :** 3004

Lister les contacts

```
GET /contacts?companyId=uuid&search=john
Authorization: Bearer <token>
```

Créer un contact

```
POST /contacts
Authorization: Bearer <token>
Content-Type: application/json

{
  "companyId": "uuid",
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@techcorp.com",
  "position": "CTO",
  "phone": "+33123456789",
  "linkedinUrl": "https://linkedin.com/in/johndoe"
}
```

Entretiens

Base URL : /interviews **Port direct :** 3005

Lister les entretiens

```
GET /interviews?upcoming=true&date=2025-01-27
Authorization: Bearer <token>
```

Planifier un entretien

```
POST /interviews
Authorization: Bearer <token>
Content-Type: application/json

{
  "applicationId": "uuid",
  "contactIds": ["uuid1", "uuid2"],
  "scheduledAt": "2025-01-30T14:00:00Z",
  "type": "technical",
  "location": "Bureau Paris",
  "notes": "Entretien technique React/Node.js"
}
```

Appels

Base URL : /calls **Port direct :** 3006

Enregistrer un appel

```
POST /calls
Authorization: Bearer <token>
Content-Type: application/json

{
  "contactId": "uuid",
  "type": "outbound",
  "duration": 30,
  "notes": "Discussion sur le poste",
  "outcome": "interested"
}
```

Événements

Base URL : /events **Port direct :** 3007

Créer un événement

```
POST /events
Authorization: Bearer <token>
Content-Type: application/json

{
  "title": "Relance candidature",
  "description": "Appeler le recruteur",
  "startDate": "2025-01-28T10:00:00Z",
  "eventType": "reminder",
  "entityType": "application",
  "entityId": "uuid",
  "reminderAt": "2025-01-28T09:00:00Z"
}
```

Suivi (Followups)

Base URL : /followups Port direct : 3008

Créer un suivi

```
POST /followups
Authorization: Bearer <token>
Content-Type: application/json

{
  "applicationId": "uuid",
  "contactId": "uuid",
  "type": "email",
  "scheduledAt": "2025-01-29T15:00:00Z",
  "notes": "Envoyer email de relance"
}
```

Profils

Base URL : /profiles Port direct : 3009

Mettre à jour le profil

```
PUT /profiles/me
Authorization: Bearer <token>
Content-Type: application/json
```

```
{
  "firstName": "John",
  "lastName": "Doe",
  "phone": "+33123456789",
  "preferences": {
    "theme": "dark",
    "language": "fr",
    "notifications": {
      "email": true,
      "push": true,
      "reminders": true
    }
  }
}
```

Notifications

Base URL : /notifications **Port direct :** 3010

Lister les notifications

```
GET /notifications?unread=true&limit=10
Authorization: Bearer <token>
```

Marquer comme lue

```
PUT /notifications/{id}/read
Authorization: Bearer <token>
```

Dashboard

Base URL : /dashboard **Port direct :** 3012

Vue d'ensemble

GET /dashboard/overview
Authorization: Bearer <token>

Analytics

GET /dashboard/analytics?period=30d&metrics=applications,interviews
Authorization: Bearer <token>

 Codes d'erreur

Format des erreurs

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Données invalides",
    "details": {
      "email": ["Champ requis", "Format invalide"]
    }
  },
  "timestamp": "2025-01-27T10:00:00Z"
}
```

Codes d'erreur courants

Code	HTTP Status	Description
VALIDATION_ERROR	400	Données invalides
UNAUTHORIZED	401	Authentification requise
FORBIDDEN	403	Permissions insuffisantes
NOT_FOUND	404	Ressource introuvable
CONFLICT	409	Conflit de données
RATE_LIMITED	429	Limite de requêtes atteinte

INTERNAL_ERROR	500	Erreur serveur
----------------	-----	----------------

Authentification

JWT Tokens

- **Access Token** : 1 heure de validité
- **Refresh Token** : 30 jours de validité
- **Format** : Bearer <token>

Rôles et permissions

```
{
  "user": {
    "permissions": ["read", "write_own"]
  },
  "admin": {
    "permissions": ["read", "write", "delete", "manage_users"]
  },
  "super_admin": {
    "permissions": ["read", "write", "delete", "manage_users", "system_admin"]
  }
}
```

Rate Limiting

- **Général** : 1000 requêtes/15min
- **Authentification** : 5 tentatives/5min
- **Endpoints sensibles** : 10 requêtes/min

Exemples d'utilisation

Création complète d'une candidature

```
// 1. Authentification
const loginResponse = await fetch('/auth/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    email: 'user@example.com',
    password: 'password123'
  })
});

const { accessToken } = await loginResponse.json();

// 2. Création d'une entreprise
const companyResponse = await fetch('/companies', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${accessToken}`
  },
  body: JSON.stringify({
    name: 'TechCorp',
    website: 'https://techcorp.com',
    industry: 'Technology'
  })
});

const { data: company } = await companyResponse.json();

// 3. Création d'un contact
const contactResponse = await fetch('/contacts', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${accessToken}`
  },
  body: JSON.stringify({
    companyId: company.id,
    firstName: 'Jane',
    lastName: 'Smith',
    email: 'jane.smith@techcorp.com',
    position: 'HR Manager'
  })
});

const { data: contact } = await contactResponse.json();

// 4. Création de la candidature
const applicationResponse = await fetch('/applications', {
  method: 'POST',
  headers: {
```

```
'Content-Type': 'application/json',
'Authorization': `Bearer ${accessToken}`
},
body: JSON.stringify({
  companyId: company.id,
  position: 'Développeur Full Stack',
  status: 'applied',
  notes: 'Candidature spontanée'
})
});

const { data: application } = await applicationResponse.json();

// 5. Planification d'un entretien
await fetch('/interviews', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${accessToken}`
  },
  body: JSON.stringify({
    applicationId: application.id,
    contactIds: [contact.id],
    scheduledAt: '2025-02-01T14:00:00Z',
    type: 'technical'
  })
});
```

Utilisation avec cURL


```
# Authentification
TOKEN=$(curl -s -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"user@example.com","password":"password123"}' \
  | jq -r '.data.accessToken')

# Lister les candidatures
curl -X GET "http://localhost:3000/applications" \
  -H "Authorization: Bearer $TOKEN" \
  | jq '.'

# Créer une entreprise
curl -X POST http://localhost:3000/companies \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "name": "TechCorp",
    "industry": "Technology",
    "website": "https://techcorp.com"
  }' \
  | jq '.'
```

Ressources

- [Architecture](#) - Vue technique complète
 - [Base de données](#) - Schémas et relations
 - [Services](#) - Détail des microservices
 - [Postman Collection](#) - Collection Postman
 - [OpenAPI Spec](#) - Spécification OpenAPI
-

Version: 4.1 - API étendue **Dernière mise à jour:** Octobre 2025



Endpoints API - JobbingTrack

Source: *api/endpoints.md*

Liste exhaustive de tous les endpoints disponibles dans l'API JobbingTrack v4.1.

[← Retour au README principal](#)



Vue d'ensemble

Cette section liste tous les endpoints disponibles par service avec leurs méthodes HTTP, paramètres et codes de réponse.



API Gateway Health

Endpoints système

```
GET /health
GET /metrics
GET /ready
GET /version
```



Authentification (Port 3001)

Sessions

```
POST /auth/login
POST /auth/register
POST /auth/logout
GET /auth/verify
POST /auth/refresh
POST /auth/forgot-password
POST /auth/reset-password
```

Utilisateurs

```
GET    /auth/users
GET    /auth/users/{id}
POST   /auth/users
PUT    /auth/users/{id}
DELETE /auth/users/{id}
```

Applications (Port 3002)

CRUD Applications

```
GET    /applications
GET    /applications/{id}
POST   /applications
PUT    /applications/{id}
DELETE /applications/{id}
```

Actions

```
GET    /applications/{id}/status
PUT    /applications/{id}/status
GET    /applications/{id}/history
POST   /applications/{id}/archive
POST   /applications/{id}/restore
```

Recherche et filtres

```
GET    /applications/search
GET    /applications/export
GET    /applications/statistics
```

Entreprises (Port 3003)

CRUD Companies

```
GET    /companies
GET    /companies/{id}
POST   /companies
PUT    /companies/{id}
DELETE /companies/{id}
```

Recherche

```
GET    /companies/search
GET    /companies/{id}/contacts
GET    /companies/{id}/applications
```

Contacts (Port 3004)

CRUD Contacts

```
GET    /contacts
GET    /contacts/{id}
POST   /contacts
PUT    /contacts/{id}
DELETE /contacts/{id}
```

Relations

```
GET    /contacts/{id}/companies
GET    /contacts/{id}/applications
GET    /contacts/{id}/interviews
GET    /contacts/{id}/calls
GET    /contacts/{id}/events
GET    /contacts/{id}/history
```

Entretiens (Port 3005)

CRUD Interviews

```
GET    /interviews
GET    /interviews/{id}
POST   /interviews
PUT    /interviews/{id}
DELETE /interviews/{id}
```

Planning

```
GET    /interviews/upcoming
GET    /interviews/today
GET    /interviews/calendar
POST   /interviews/{id}/reschedule
POST   /interviews/{id}/cancel
```

Notes et feedback

```
POST   /interviews/{id}/notes
GET    /interviews/{id}/feedback
PUT    /interviews/{id}/rating
```

Appels (Port 3006)

CRUD Calls

```
GET    /calls
GET    /calls/{id}
POST   /calls
PUT    /calls/{id}
DELETE /calls/{id}
```

Historique

```
GET    /calls/history
GET    /calls/scheduled
GET    /calls/missed
POST   /calls/{id}/complete
```

Événements (Port 3007)

CRUD Events

```
GET    /events
GET    /events/{id}
POST   /events
PUT    /events/{id}
DELETE /events/{id}
```

Calendrier

```
GET    /events/calendar
GET    /events/upcoming
GET    /events/today
GET    /events/month/{year}/{month}
```

Suivi (Port 3008)

CRUD Followups

```
GET    /followups
GET    /followups/{id}
POST   /followups
PUT    /followups/{id}
DELETE /followups/{id}
```

Actions

```
POST    /followups/{id}/complete
POST    /followups/{id}/postpone
GET     /followups/due
GET     /followups/overdue
```

Profils (Port 3009)

Profil utilisateur

```
GET     /profiles/me
PUT     /profiles/me
POST    /profiles/avatar
DELETE  /profiles/avatar
```

Préférences

```
GET     /profiles/preferences
PUT     /profiles/preferences
GET     /profiles/settings
PUT     /profiles/settings
```

Notifications (Port 3010)

CRUD Notifications

```
GET     /notifications
GET     /notifications/{id}
PUT     /notifications/{id}/read
DELETE  /notifications/{id}
```

Paramètres

```
GET    /notifications/settings
PUT    /notifications/settings
POST   /notifications/test
```

Workflows (Port 3011)

CRUD Workflows

```
GET    /workflows
GET    /workflows/{id}
POST   /workflows
PUT    /workflows/{id}
DELETE /workflows/{id}
```

Exécution

```
POST   /workflows/{id}/execute
GET    /workflows/{id}/runs
GET    /workflows/{id}/logs
```

Dashboard (Port 3012)

Analytics

```
GET    /dashboard/overview
GET    /dashboard/analytics
GET    /dashboard/metrics
GET    /dashboard/statistics
```

Rapports


```
GET    /dashboard/reports
POST   /dashboard/reports
GET    /dashboard/reports/{id}
```

Export

```
POST   /dashboard/export
GET    /dashboard/export/{id}
```



Sécurité (Port 3013)

Audit

```
GET    /security/audit
GET    /security/audit/{id}
POST   /security/audit/search
```

Alertes

```
GET    /security/alerts
POST   /security/alerts
PUT    /security/alerts/{id}
DELETE /security/alerts/{id}
```

Sessions

```
GET    /security/sessions
DELETE /security/sessions/{id}
POST   /security/sessions/revoke-all
```



Métriques système (Port 3018)

Métriques

```
GET    /system-metrics
GET    /system-metrics/{service}
GET    /system-metrics/history
GET    /system-metrics/performance
```

Alertes

```
GET    /system-metrics/alerts
POST   /system-metrics/alerts
PUT    /system-metrics/alerts/{id}
DELETE /system-metrics/alerts/{id}
```



Déploiement (Port 3016)

Status

```
GET    /deployment/status
GET    /deployment/services
GET    /deployment/health
```

Actions

```
POST   /deployment/deploy
POST   /deployment/rollback
POST   /deployment/restart
POST   /deployment/stop
POST   /deployment/start
```

Historique

```
GET    /deployment/history
GET    /deployment/history/{id}
GET    /deployment/logs
```

Docker Stats (Port 3015)

Statistiques

```
GET    /docker-stats
GET    /docker-stats/containers
GET    /docker-stats/images
GET    /docker-stats/volumes
GET    /docker-stats/networks
```

Monitoring

```
GET    /docker-stats/monitoring
GET    /docker-stats/resources
GET    /docker-stats/performance
```

Codes de réponse HTTP

Réponses de succès

- **200 OK** : Requête réussie
- **201 Created** : Ressource créée
- **204 No Content** : Succès sans contenu

Erreurs client

- **400 Bad Request** : Requête malformée
- **401 Unauthorized** : Authentification requise
- **403 Forbidden** : Accès refusé
- **404 Not Found** : Ressource introuvable
- **409 Conflict** : Conflit de données
- **422 Unprocessable Entity** : Validation échouée

- **429 Too Many Requests** : Limite atteinte

Erreurs serveur

- **500 Internal Server Error** : Erreur serveur
- **502 Bad Gateway** : Service indisponible
- **503 Service Unavailable** : Maintenance
- **504 Gateway Timeout** : Timeout

Filtres et paramètres

Pagination

```
?page=1&limit=20&sort=createdAt&order=desc
```

Filtres

```
?status=applied&companyId=123&dateFrom=2025-01-01&dateTo=2025-12-31
```

Recherche

```
?search=term&q=query&fields=name,description
```

Ressources

- [API Reference complète](#) - Guide détaillé
- [Architecture](#) - Vue technique
- [Postman Collection](#)
- [OpenAPI Specification](#)

Version: 4.1 - Endpoints complets **Dernière mise à jour:** Octobre 2025

Démarrage Rapide - JobbingTrack

Source: *deployment/getting-started.md*

Guide d'installation et configuration pour JobbingTrack v4.1.

[← Retour au README principal](#)

Prérequis

Configuration système minimale

- **CPU** : 2 cœurs
- **RAM** : 4GB
- **Stockage** : 10GB disponibles
- **OS** : Linux/macOS/Windows (WSL2 recommandé)

Outils requis

- **Docker** : 20.10+
 - **Docker Compose** : 2.0+
 - **Git** : 2.30+
 - **Node.js** : 20 LTS (pour développement)
 - **Make** (optionnel, pour les commandes automatisées)
-

Installation rapide

1. Clonage du projet

```
git clone https://github.com/votre-repo/jobbingtrack.git
cd jobbingtrack
```

2. Configuration de l'environnement

```
# Copier le fichier d'environnement
cp .env.example .env

# Éditer les variables d'environnement
nano .env
```

3. Démarrage des services

```
# Services de base (PostgreSQL, Redis, API Gateway, Frontend)
make up

# Ou avec Docker Compose directement
docker-compose up -d postgres redis api-gateway frontend
```

4. Vérification de l'installation

```
# Vérifier que tous les services sont démarrés
curl http://localhost:3000/health

# Accéder à l'interface web
open http://localhost:8080

# Vérifier les logs
make logs
```

Architecture de démarrage

Services démarrés par défaut

```
make up # Démarre automatiquement :
```

- ├─  PostgreSQL (5432) - Base de données
- ├─  Redis (6379) - Cache et sessions
- ├─  API Gateway (3000) - Point d'entrée API
- ├─  Frontend (8080) - Interface web
- ├─  Metrics Aggregator (3014) - Monitoring
- └─  cAdvisor (8081) - Métriques Docker

Services optionnels

```
# Ajouter les services métier
make up-profile PROFILE=auth           # Authentification
make up-profile PROFILE=applications  # Candidatures
make up-profile PROFILE=companies     # Entreprises
make up-profile PROFILE=contacts      # Contacts
make up-profile PROFILE=interviews    # Entretiens
make up-profile PROFILE=calls         # Appels
make up-profile PROFILE=events        # Événements
make up-profile PROFILE=followups     # Suivi
make up-profile PROFILE=profiles      # Profils
make up-profile PROFILE=notifications # Notifications
make up-profile PROFILE=workflows     # Workflows

# Services système
make up-profile PROFILE=dashboard     # Dashboard admin
make up-profile PROFILE=security       # Sécurité
make up-profile PROFILE=monitoring    # Monitoring complet
make up-profile PROFILE=full          # Tous les services
```



Configuration avancée

Variables d'environnement

Base de données

```
POSTGRES_DB=jobbingtrack
POSTGRES_USER=jobbingtrack
POSTGRES_PASSWORD=jobbingtrack123
DATABASE_URL=postgresql://jobbingtrack:jobbingtrack123@postgres:5432/jobbingtrack
```

Authentification

```
JWT_SECRET=your-super-secret-jwt-key-change-in-production-2025
JWT_REFRESH_SECRET=your-refresh-token-secret-change-in-production-2025
JWT_EXPIRES_IN=1h
JWT_REFRESH_EXPIRES_IN=30d
```

Services

```

REDIS_URL=redis://redis:6379
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8080
FRONTEND_URL=http://localhost:8080

```

Email (SMTP)

```

SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-email@gmail.com
SMTP_PASS=your-app-password
SMTP_FROM=JobbingTrack <noreply@jobbingtrack.com>

```

Configuration Docker

Réseau personnalisé

```

# docker-compose.yml
networks:
  jobbingtrack-network:
    driver: bridge
    ipam:
      config:
        - subnet: 172.20.0.0/16

services:
  postgres:
    networks:
      - jobbingtrack-network
# ... autres configurations

```

Volumes persistants

```

volumes:
  postgres_data:
    driver: local
  redis_data:
    driver: local
  uploads:
    driver: local

```


Tests et validation

Tests automatiques

```
# Tests unitaires
make test

# Tests d'intégration
make test-integration

# Tests end-to-end
make test-e2e

# Tests de performance
make test-performance
```

Vérifications manuelles

1. Vérifier la base de données

```
# Connexion PostgreSQL
docker exec -it jobbingtrack_postgres_1 psql -U jobbingtrack -d jobbingtrack

# Lister les schémas
\dn

# Lister les tables principales
SELECT schemaname, tablename FROM pg_tables WHERE schemaname IN ('auth', 'applicat
```

2. Vérifier les services

```
# Health check API Gateway
curl http://localhost:3000/health

# Métriques Prometheus
curl http://localhost:9090/api/v1/query?query=up

# Interface cAdvisor
open http://localhost:8081
```

3. Vérifier le frontend

```
# Build du frontend
cd frontend && npm run build

# Tests frontend
cd frontend && npm run test

# Linting
cd frontend && npm run lint
```



Applications mobiles

Flutter Mobile App

```
# Démarrer l'émulateur mobile
make up-profile PROFILE=mobile

# Ou directement
docker-compose -f flutter-mobile-app/docker-compose.yml up -d

# Accéder à l'interface mobile
open http://localhost:8090
```

Configuration mobile

```
// lib/services/api_service.dart
class ApiService {
  static const String baseUrl = 'http://localhost:3000';
  static const String wsUrl = 'ws://localhost:3000';

  // Configuration pour production
  // static const String baseUrl = 'https://api.jobbingtrack.com';
  // static const String wsUrl = 'wss://api.jobbingtrack.com';
}
```

Dépannage

Problèmes courants

1. Ports déjà utilisés

```
# Vérifier les ports occupés
lsof -i :3000
lsof -i :8080

# Modifier les ports dans docker-compose.yml
# ports:
#   - "3001:3000" # API Gateway
#   - "8081:8080" # Frontend
```

2. Base de données non accessible

```
# Vérifier les logs PostgreSQL
make logs SERVICE=postgres

# Reset de la base de données
make down
docker volume rm jobbingtrack_postgres_data
make up
```

3. Services qui ne démarrent pas

```
# Vérifier les logs
make logs

# Redémarrer les services
make restart

# Rebuild des images
make build
make up
```

4. Problèmes de réseau

```
# Vérifier le réseau Docker
docker network ls
docker network inspect jobbingtrack-network

# Recréer le réseau si nécessaire
docker-compose down
docker network rm jobbingtrack-network
make up
```

Prochaines étapes

1. Configuration complète

- [Guide de développement](#)
- [Documentation API](#)
- [Guide d'administration](#)

2. Déploiement en production

- [Guide production](#)
- [Configuration sécurité](#)
- [Monitoring et métriques](#)

3. Personnalisation

- [Configuration frontend](#)
- [Application mobile](#)
- [Thèmes et personnalisation](#)

Support

- **Logs** : `make logs` pour voir tous les logs
- **Monitoring** : `http://localhost:8081` (cAdvisor)
- **Santé** : `http://localhost:3000/health`
- **Tests** : `make test` pour valider l'installation

Version: 4.1 - Installation simplifiée **Dernière mise à jour:** Octobre 2025

Déploiement Production - JobbingTrack

Source: *deployment/production.md*

Guide de déploiement en production pour JobbingTrack v4.1.

[← Retour au README principal](#)

Vue d'ensemble

Configuration et déploiement sécurisé en environnement de production.

Version: 4.1 - Déploiement production **Dernière mise à jour:** Octobre 2025

Sécurité Déploiement - JobbingTrack

Source: *deployment/security.md*

Configuration sécurité pour le déploiement de JobbingTrack v4.1.

[← Retour au README principal](#)

Vue d'ensemble

Sécurisation de l'infrastructure et des communications.

Version: 4.1 - Sécurité déploiement **Dernière mise à jour:** Octobre 2025



Configuration Développement - JobbingTrack

Source: *development/setup.md*

Guide de configuration de l'environnement de développement pour JobbingTrack v4.1.

[← Retour au README principal](#)



Vue d'ensemble

Configuration complète pour développer sur JobbingTrack avec Node.js, TypeScript, Prisma, Docker et les tests.



Prérequis

Outils système

- **Node.js** : 20 LTS
- **npm** : 8.0+ ou **yarn** : 1.22+
- **Docker** : 20.10+
- **Docker Compose** : 2.0+
- **Git** : 2.30+

IDE recommandé

- **Visual Studio Code** avec extensions :
 - ES7+ React/Redux/React-Native snippets
 - Prettier
 - ESLint
 - Docker
 - Prisma
-



Configuration rapide

1. Installation des outils

```
# Node.js 20 LTS
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

# Docker
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# Docker Compose
sudo apt-get install docker-compose-plugin
```

2. Clonage et installation

```
# Clonage du projet
git clone https://github.com/votre-repo/jobbingtrack.git
cd jobbingtrack

# Installation des dépendances
npm install

# Configuration de l'environnement
cp .env.example .env
nano .env
```

3. Démarrage en mode développement

```
# Services de développement
make dev

# Ou manuellement
docker-compose -f docker-compose.dev.yml up -d

# Installation des dépendances frontend
cd frontend && npm install
cd ../backend && npm install

# Démarrage du frontend en mode dev
cd frontend && npm run dev

# Démarrage des services backend
cd backend && npm run dev:services
```




Configuration détaillée

Variables d'environnement

Développement (.env)

```
# Base de données de développement
DATABASE_URL=postgresql://jobbingtrack:jobbingtrack123@localhost:5432/jobbingtrack

# JWT pour le développement (moins sécurisé mais pratique)
JWT_SECRET=dev-jwt-secret-key-2025
JWT_REFRESH_SECRET=dev-refresh-secret-2025

# Services de développement
REDIS_URL=redis://localhost:6379
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8080,http://localhost:3001

# Frontend
FRONTEND_URL=http://localhost:8080
NEXT_PUBLIC_API_URL=http://localhost:3000

# Email (optionnel en dev)
SMTP_HOST=localhost
SMTP_PORT=1025
SMTP_USER=
SMTP_PASS=

# Logs
LOG_LEVEL=debug
NODE_ENV=development
```

Production (.env.production)

```
DATABASE_URL=postgresql://jobbingtrack:secure_password@postgres:5432/jobbingtrack
JWT_SECRET=your-production-secret-key-2025
JWT_REFRESH_SECRET=your-production-refresh-secret-2025
REDIS_URL=redis://redis:6379
ALLOWED_ORIGINS=https://yourdomain.com,https://app.yourdomain.com
FRONTEND_URL=https://yourdomain.com
SMTP_HOST=smtp.sendgrid.net
SMTP_PORT=587
SMTP_USER=apikey
SMTP_PASS=your-sendgrid-api-key
LOG_LEVEL=info
NODE_ENV=production
```

Configuration Docker Compose pour le développement

docker-compose.dev.yml

```

version: '3.8'
services:
  postgres-dev:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: jobbingtrack_dev
      POSTGRES_USER: jobbingtrack
      POSTGRES_PASSWORD: jobbingtrack123
    ports:
      - "5433:5432"
    volumes:
      - postgres_dev_data:/var/lib/postgresql/data
    command: postgres -c log_statement=all -c log_destination=stderr

  redis-dev:
    image: redis:7-alpine
    ports:
      - "6380:6379"
    command: redis-server --appendonly yes

# Services backend avec hot reload
auth-service-dev:
  build:
    context: ./backend/auth-service
    dockerfile: Dockerfile.dev
  volumes:
    - ./backend/auth-service/src:/app/src
    - ./backend/auth-service/nodemon.json:/app/nodemon.json
  environment:
    NODE_ENV: development
  ports:
    - "3001:3000"

# Frontend avec hot reload
frontend-dev:
  build:
    context: ./frontend
    dockerfile: Dockerfile.dev
  volumes:
    - ./frontend/src:/app/src
    - ./frontend/public:/app/public
  ports:
    - "8080:3000"
  environment:
    NODE_ENV: development

```

Dockerfile.dev pour les services

```
FROM node:20-alpine

WORKDIR /app

# Installation des dépendances
COPY package*.json ./
RUN npm install

# Configuration nodemon pour hot reload
COPY nodemon.json ./
RUN npm install -g nodemon

# Copier le code source
COPY . .

EXPOSE 3000

# Démarrage en mode développement
CMD ["nodemon", "src/server.js"]
```

Tests et qualité

Configuration des tests

Jest (Backend)

```
// backend/jest.config.js
module.exports = {
  preset: 'ts-jest',
  testEnvironment: 'node',
  roots: ['<rootDir>/src', '<rootDir>/tests'],
  testMatch: ['**/__tests__/**/*.ts', '**/?(*.)+(spec|test).ts'],
  transform: {
    '^.+\\.ts$': 'ts-jest',
  },
  collectCoverageFrom: [
    'src/**/*.ts',
    '!src/**/*.d.ts',
    '!src/server.ts',
  ],
  coverageDirectory: 'coverage',
  coverageReporters: ['text', 'lcov', 'html'],
  setupFilesAfterEnv: ['<rootDir>/tests/setup.ts'],
  testTimeout: 10000,
};
```

Playwright (Frontend)

```
// frontend/playwright.config.ts
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 1 : undefined,
  reporter: 'html',
  use: {
    baseURL: 'http://localhost:8080',
    trace: 'on-first-retry',
  },
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
  ],
  webServer: {
    command: 'npm run build && npm run preview',
    url: 'http://localhost:8080',
    reuseExistingServer: !process.env.CI,
  },
});
```

Commandes de test

```
# Tests backend (tous services)
make test

# Tests frontend
cd frontend && npm run test

# Tests d'intégration
make test-integration

# Tests E2E avec Playwright
make test-e2e

# Tests de performance
make test-performance

# Coverage
make coverage
```

Linting et formatage

```
# Linting backend
cd backend && npm run lint

# Linting frontend
cd frontend && npm run lint

# Formatage automatique
make format

# Type checking
cd frontend && npm run type-check
```



Workflow de développement

Git Hooks (Husky)

```
# Installation des hooks
cd frontend && npm run prepare

# Hooks configurés :
# - pre-commit: linting et tests
# - pre-push: tests complets
# - commit-msg: format des messages
```

Développement avec Docker

```
# Démarrage en mode développement
make dev

# Services avec hot reload
docker-compose -f docker-compose.dev.yml up

# Build des services
make build-dev

# Logs en temps réel
make logs-dev
```

Développement sans Docker

```
# Base de données locale
sudo -u postgres createdb jobbingtrack_dev
sudo -u postgres psql -c "CREATE USER jobbingtrack WITH PASSWORD 'jobbingtrack123'"
sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE jobbingtrack_dev TO jobl"

# Démarrage des services en local
cd backend && npm run dev:local

# Frontend en local
cd frontend && npm run dev
```

Debugging

Logs de développement


```
# Logs de tous les services
make logs

# Logs d'un service spécifique
make logs SERVICE=auth-service

# Logs avec suivi
make logs-follow

# Logs du frontend
cd frontend && npm run logs
```

Outils de debugging

VS Code Debug Configuration

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Debug Auth Service",
      "type": "node",
      "request": "launch",
      "program": "${workspaceFolder}/backend/auth-service/src/server.js",
      "skipFiles": ["<node_internals>/**"],
      "env": {
        "NODE_ENV": "development",
        "DATABASE_URL": "postgresql://localhost:5433/jobbingtrack_dev"
      }
    },
    {
      "name": "Debug Frontend",
      "type": "node",
      "request": "launch",
      "program": "${workspaceFolder}/frontend/node_modules/.bin/next",
      "args": ["dev"],
      "cwd": "${workspaceFolder}/frontend"
    }
  ]
}
```

Postman pour les tests API

```
# Collection Postman disponible dans docs/api/  
# Variables d'environnement :  
# - baseUrl: http://localhost:3000  
# - authToken: Bearer <token>
```

Métriques de développement

```
# Métriques Prometheus locales  
curl http://localhost:9090/api/v1/query?query=up  
  
# Health checks  
curl http://localhost:3000/health  
curl http://localhost:3001/health  
curl http://localhost:8080/api/health
```

Performance et optimisation

Bundle Analyzer

```
# Frontend bundle analysis  
cd frontend && npm run analyze  
  
# Backend bundle analysis  
cd backend && npm run bundle-analyze
```

Profiling

```
# Node.js profiling  
node --prof src/server.js  
node --prof-process isolate-*.log > profile.txt  
  
# Frontend performance  
cd frontend && npm run lighthouse
```



Ressources de développement

Documentation technique

- [Architecture microservices](#)
- [API Reference](#)
- [Base de données](#)
- [Tests et qualité](#)

Outils de développement

- [Makefile documentation](#)
- [Scripts d'automatisation](#)
- [Configuration CI/CD](#)

Communauté et support

- [Contributing Guide](#)
- [Code of Conduct](#)
- [Issues GitHub](#)

Version: 4.1 - Environnement de développement **Dernière mise à jour:** Octobre 2025



Workflow Développement - JobbingTrack

Source: *development/workflow.md*

Guide du workflow de développement pour JobbingTrack v4.1.

[← Retour au README principal](#)



Vue d'ensemble

Processus complet de développement, de la conception à la production.



Ressources

- [Configuration développement](#) - Environnement de développement
 - [Tests et qualité](#) - Stratégies de tests
 - [Architecture](#) - Vue technique
-

Version: 4.1 - Workflow de développement **Dernière mise à jour:** Octobre 2025

Tests et Qualité - JobbingTrack

Source: *development/testing.md*

Guide des tests et de la qualité du code pour JobbingTrack v4.1.

[← Retour au README principal](#)

Vue d'ensemble

Stratégies de tests complètes : unitaires, d'intégration, E2E et performance.

Ressources

- [Configuration développement](#) - Environnement de développement
 - [Workflow développement](#) - Processus de développement
-

Version: 4.1 - Tests et qualité **Dernière mise à jour:** Octobre 2025



Guide Frontend - JobbingTrack

Source: *frontend/guide.md*

Guide de développement frontend pour JobbingTrack v4.1.

[← Retour au README principal](#)



Vue d'ensemble

Développement de l'interface Next.js avec TypeScript, Tailwind CSS et Radix UI.

Version: 4.1 - Guide frontend **Dernière mise à jour:** Octobre 2025



Guide Mobile - JobbingTrack

Source: *mobile/guide.md*

Guide de développement de l'application mobile Flutter pour JobbingTrack v4.1.

[← Retour au README principal](#)



Vue d'ensemble

Application mobile cross-platform avec Flutter, synchronisation temps réel et mode offline.

Version: 4.1 - Guide mobile **Dernière mise à jour:** Octobre 2025



Guide Administration - JobbingTrack

Source: *administration/guide.md*

Guide d'administration et dashboard pour JobbingTrack v4.1.

[← Retour au README principal](#)



Vue d'ensemble

Interface d'administration complète pour la gestion du système.

Version: 4.1 - Guide administration **Dernière mise à jour:** Octobre 2025



Guide Dépannage - JobbingTrack

Source: *troubleshooting/guide.md*

Guide de résolution des problèmes pour JobbingTrack v4.1.

[← Retour au README principal](#)



Vue d'ensemble

Solutions aux problèmes courants et diagnostic.

Version: 4.1 - Guide dépannage **Dernière mise à jour:** Octobre 2025

Guide Performance - JobbingTrack

Source: *performance/guide.md*

Guide d'optimisation des performances pour JobbingTrack v4.1.

[← Retour au README principal](#)

Vue d'ensemble

Optimisations, monitoring et bonnes pratiques performance.

Version: 4.1 - Guide performance **Dernière mise à jour:** Octobre 2025

Guide Sécurité - JobbingTrack

Source: *security/guide.md*

Guide de sécurité et bonnes pratiques pour JobbingTrack v4.1.

[← Retour au README principal](#)

Vue d'ensemble

Configuration sécurité, authentification et protection.

Version: 4.1 - Guide sécurité **Dernière mise à jour:** Octobre 2025
